Stock Selection Based on Kernel Principal Component Analysis

HUANG POYUAN

April 27, 2021

0.1 Introduction

This project investigate how Kernel Principal Component Analysis (KPCA) reduce variables in our predicting model so as to make further portfolio management. The motivation comes from the drawback of using PCA on large dataset where all SP500 or NASDAQ tickers are included in our universe. PCA deals with linear classification whereas KPCA deals with non-linear classification and have even more versatile decision boundaries based on the choice of kernel function. In this paper, I will introduce the method of KPCA and then the trading strategy using it. Also, I will present empirical studies and analysis over the SP500 top30 stocks daily dataset.

0.2 Literature Review

In the Quantitative Stock Selection Strategies Based on Kernel Principal Component Analysis by Meiyi Zhou*, Lianqian Yin, they've used kernel PCA to build predictive models stock returns and experimented a various of kernel functions. They've also constructed and make robustness analysis on their portfolio in "back-view" method by simulating stock process and looked at indicators such as shape ratio, beta..etc. The result of this paper showed the effectiveness of kernel PCA capturing market information and return prediction within a descent number of factors and ended up a 41 percentage excess return on Shanghai and Shenzhen 300 index data sets.

0.3 Kernel PCA

The idea of using principal component analysis on is that one wish to extract only the important factors that explained moments the most of our universe. Although one might lose some information or variance per se about the market, one does not need to include all factors in the predicting model that will cause statistically insignificant or collinearity of the parameters.
In the case of having T ¡¡ N dataset where T is the number of observation and N is the number of features, PCA on the stock covariance matrix may be tricky as it no longer becomes full rank. However, KPCA maps the original space to higher dimension to find a suitable hyper-plane for classification. The kernel function does the mapping for us. Once we get kernel matrix, we decompose the matrix as we did on covriance matrix.

- Centered data $\sum h(x) = 0$
- Compute kernel matrix K by kernel function $< h(x_i), h(x_j) >= K(x_i, x_j)$
- Decompose K to get eigen-values and eigen-vectors

- Set threshold of variance explained to determine the number of extracted factors
- Project data to eigen-vectors to get principal components.

0.3.1 Kernel functions

Kernel function is a similarity measure. The optimal kernel function selection can be done by cross-validation or prior knowledge of dataset. Here we only introduce radial basis function (RBF) and polynomial (POLY) function.

- POLY of degree d: $K(x_i, x_j) = (x_i^T x_j + 1)^d$
- RBF with tuning γ: $K(x_i, x_j) = exp(-\gamma||x_i - x_j||_2^2)$

In general, RBF can map to infinite dimensions and its decision boundaries are more flexible. POLY has better performance in non-linear classification and is less time consuming. As for the downside, RBF may be hard to keep track of data after transformation whereas POLY can easily over-fit as degree raises.

0.3.2 Comparison between RBF vs POLY

The dataset are top80 SP500 from 2009-12-31 to 2020-10-30 and is regrouped monthly. The training window is 24 (months). The statistical summary of predicted model are shown below.

	RBF (gamma)		**POLY (d)**	
Parameters	1	2	3	4
1st Explained Var	0.239	0.181	0.412	0.408
Accumulated Var	0.447	0.376	0.619	0.616

POLY is better than RBF in terms of explained variance. However, POLY shows a potential over-fit since the explained variance does not increase much as the degree raise. And BRF has balanced explained variance over four components and that accumulated variance increase as smaller gamma is used.

0.3.3 Demo of explained variance

Explained variance shows how well the model captures the market information. As kPCA increase its number of factors, explained variance increases non-decreasingly . I select 20 factors with RBF gamma 0.03 for demonstration. (Note: This is presented in demo.py)

```
___Cumulative Explined Variance Ratio___
[0.26915766 0.33065968 0.37927474 0.41925087 0.45410776 0.48663154
 0.51763351 0.5447666  0.56860465 0.59090259 0.61202246 0.63237629
 0.65155425 0.6693139  0.68687787 0.70327284 0.71858934 0.73231086
 0.74598254 0.75862951]

                                 OLS Regression Results
=======================================================================================
Dep. Variable:                    SPX   R-squared (uncentered):                  0.951
Model:                            OLS   Adj. R-squared (uncentered):             0.942
Method:                 Least Squares   F-statistic:                             105.6
Date:                Tue, 08 Dec 2020   Prob (F-statistic):                   2.79e-62
Time:                        11:10:15   Log-Likelihood:                         426.82
No. Observations:                 130   AIC:                                    -813.6
Df Residuals:                     110   BIC:                                    -756.3
Df Model:                          20
Covariance Type:            nonrobust
```

Figure 1: explained variance

0.3.4 Cross-Validation tuning parameter

An optimal choice of kernel parameter will improve the classification precision for the first few components. Here we use GridSearchCV from "sklearn" and use MSE as our scoring function. The optimal kernel is RBF with gamma 0.03. Part of the code:

```
param_grid = [{
            "gamma": np.linspace(0.03, 4, 20),
            "kernel": ["rbf","poly" ]
            }]

kpca=KernelPCA(fit_inverse_transform=True, n_jobs=-1)  # n_jobs using how many parallel processor
grid_search = GridSearchCV(kpca, param_grid, cv=3, scoring=MSE) #cv: how many fold
```

Figure 2: CV

0.4 Methodology

Given window (wd), numbers of tickers (N), number of factors extracted (p), active weights of each stock (aw_i).

- Select data window Data[wd] and centered the data X
- Do KPCA to get $PC_i = X\frac{v_i}{\sqrt{\lambda_i}}$ up to p
- Construct multi-factor model $r_i = \alpha_i + \sum_{j=1}^{p} \beta_{ij} PC_j$
- Predict r_{it+1} based on the t th projected PC
- Select stocks that have higher predicted return than the benchmark
- Select more stocks that have less correlation to benchmark if the benchmark is negative.

- Use mean-variance optimization to get optimal aw_i
- Execute asset allocation based on the weights and roll the window

0.4.1 Mean-Variance Optimization

Based on long only and beta neutral policy, we can quadratic programming with restrictions. Let $x = [\Delta x_1, \Delta x_2, ..\Delta x_n]$ be the active weights for the selected stocks. λ is risk-aversion parameter and μ is mean return. $\beta = [\beta_1, \beta_2, ..\beta_n]$ is beta of each stock. We use scipy optimize package.

$$
\begin{aligned}
\underset{X}{\text{minimize}} \quad & \frac{\lambda}{2} x^{'} \Sigma x - \mu^{'} x \\
\text{subject to} \quad & 1^{'} x = 0 \\
& \beta^{'} x = 0 \\
& 1^{'} x \succeq w_b \\
& \sqrt{x' \Sigma x} \leq te_d
\end{aligned}
$$

```
arg_fun = (dt, mu)
arg_cos0 = (I, 0.0)  # aw = 0
arg_cos1 =(beta, b0)   #beta neutral
arg_cos2 = (S, track)  #tracking error
bnds = ((-1/n, 1.0001),) * len(dt) # long only
cons = (
       {'type': 'eq', 'fun': cons0, 'args': arg_cos0},
       {'type': 'eq', 'fun': cons1, 'args': arg_cos1},
       {'type': 'ineq', 'fun': cons2, 'args': arg_cos2}
       )

guess = (1/n,) * len(dt)
result = scipy.optimize.minimize(fun, x0 = guess, method = 'SLSQP',
                                 args = arg_fun, constraints = cons,
                                 bounds=bnds,
                                 options={'maxiter': 101})
```

Figure 3: Optimization

0.5 Empirical Study

0.5.1 Dataset

The data set is derived from yfinance. The universe is top 80 stocks and GSPC daily adjusted close price starting from 2009-12-31 to 2020-10-30.
Each time, we select window size of 24 days x 80 to construct our model. We start our initial CASH = 10000 and no additional cash are invested afterwards.

RBF gamma 0.03 is used.

0.5.2 Result

	Portfolio	EqualWeight	Bench
End of Return(%)	635	0	207
Av. Annual TE	0.07		
Worst year of TE	2020		
Best year of TE	2013		
Av. Annual IR	1.29		
Worst year of IR	2016		
Best year of IR	2013		
Overall IC	0.69		

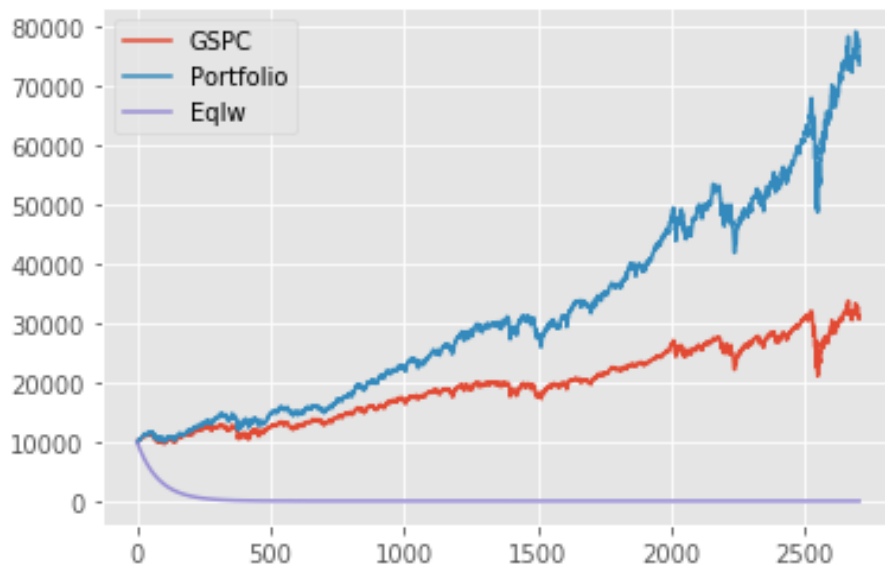

Figure 4: RBF

Note that 2020 is calculated up to October and the date of IR/TE is shifted since I put them on the start of next year.

0.5.3 IR/IC/TE Analysis

The average TE of the portfolio is 7 % which is higher than our 3 % goal. One technical reason is that the optimization did not converge given the number of iterations. Thus portfolio will only hold equal weight for that day. The second possible reason is the bias from the choice of our universe. The data set

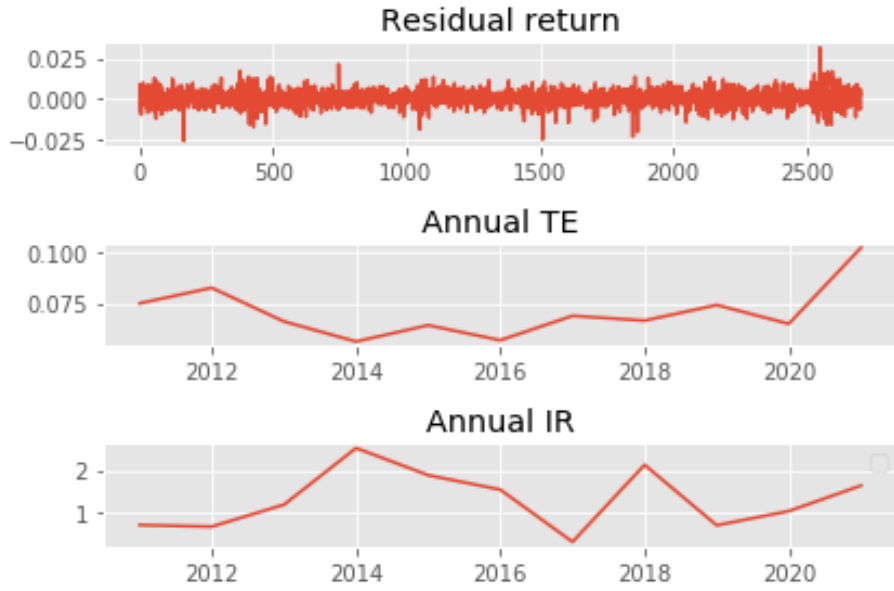

Figure 5: TE/IR

of "top80" stocks somehow uses future information while I start my portfolio construction in 2010. This is why my portfolio grow abnormally in the last few years. The third possible reason is that during back-testing, making withdraw or deposit are not allowed. Only present (value) cash is used for next asset allocation. That is also why the equal weight strategy died out way before the tech-stocks it held grow in the latest few years.

For IR, based on the formula, it should have reverse relation with TE, when TE goes down, IR will be be accumulating over time, however in the latest few years, IR does not reflect properly as the portfolio return grows abnormally.

For IC, since my stock picking rule is comparing to bench mark, I took expectation of my portfolio return having the same sign as GSPC. The result is 0.69 which slightly better than random guess (0.5). This indicated my prediction model is valid and can still be improved.

0.6 Summary/Future work

In retrospect reading the paper, technical factors, evaluation factors and macroeconomic factor...etc are also included to have a higher confidence picking to-grow stocks. The advantage using KPCA in my project is that in average, 37 stocks are being concerned instead of the whole universe of 30 stocks. This helps me to

reduce computation when doing daily optimization since less stocks data set are included. KPCA is an efficient way of dimension reduction, however it has an implied drawback of not being able to explain the extracted factors which the paper did not mention it but elegantly use it on prediction. Despite the feasibility and math support of this technique, factor analysis will be crucial part in the future work since as a portfolio manager, being able to explain the factors logically to the clients is important as well.

Another future work would be finding a faster algorithm for weight optimization since under provided constraints, sometimes the max number of iteration are met before it converged during back-testing.

0.7 Code file

There are four files. FINAL-PROJECT.py is the main file. The code is running on python Spyder. Any computer environment should be fine just make sure all packages are installed.

- FINAL-PROJECT.py: Main script
- kPCA-algo.py: The main implementation of kPCA and methodology
- cool.py: Optimizer given the constraints in class
- Backtest.py: Simple back test to support kPCA-algo
- demo.py: Simple demo on monthly data and how kPCA works.

0.8 Reference

https://www.scirp.org/journal/paperinformation.aspx?paperid=98781
https://faculty.washington.edu/ezivot/econ589/estimatingstatisticalfactormodels.pdf
https://web.stanford.edu/ boyd/cvxbook/bv$_{c}vxbook.pdf$
https://www.sciencedirect.com/science/article/pii/S1877050918301236